Relatório Trabalho Banco de Dados

Thamiris Yamate Fischer
GRR: 20243604

Sergio Sivonei Sant'Ana Filho
GRR: 20242337

Giuliano Thiago Pinheiro Tavares
GRR20240305

1. Definições

Seja U o universo de atributos e F o conjunto das dependências funcionais, segue abaixo as definições de fecho, cobertura mínima, chave candidata e normas 3FN e BCNF.

1.1 Fecho de Atributos

Dados U, F e um subconjunto de atributos $X \subseteq U$, o fecho de X em relação a F, denotado por X^+, é um conjunto de atributos $Y \subseteq U$ determinados por X.

1.2 Cobertura Mínima

Um conjunto de dependências funcionais F' é cobertura mínima de F se F' for equivalente à F, ou seja, se todas as dependências funcionais de F puderem ser deduzidas de F' e vice e versa, além de satisfazer as seguintes condições:

- Possuir um único atributo do lado direito (RHS) nas dependências;
- Não existir atributos redundantes do lado esquerdo (LHS);
- Não existir nenhuma dependência funcional redundante.

1.3 Chaves Candidatas

Dados U e F, um conjunto de atributos $X \subseteq U$ é uma chave candidata se cumprir as seguintes propriedades:

- X deve ser superchave, ou seja, $X^+ = U$;
- Nenhum subconjunto de X é superchave.

1.4 Verificação BCNF e 3FN

Dado F, as dependências funcionais, chamadas de relações, estão em 3FN se essas estiverem em 2FN e não possuem dependências transitivas, ou seja, se para todo $X \rightarrow A$, X é uma superchave ou A é um atributo primo (faz parte de uma chave candidata). Já, as relações estão em BCNF se para toda dependência funcional $X \rightarrow Y$, em F, X for superchave.

2. Decisões de Projeto

2.1 Leitura das operações

Os comandos para executar o cálculo do fecho e os demais diferem, pois, as outras operações seguem o seguinte formato `fdtool <operação> --fds <arquivo.fds>` já, no fecho, temos a flag `--X` e um atributo (`fdtool closure --fds <arquivo.fds> --X <atributos>`), para lidar com essa diferença há duas funções: `handle_closure` que lê o comando e executa a função `closure` e `handle_standards` o qual também lê o comando, mas identifica a operação solicitada e executa a função correspondente.

2.2 Representação dos Atributos e das Dependências Funcionais

Os atributos são representados como vetores de booleano com 26 posições, ou seja, as letras do alfabeto. No código, a `struct fd_t` representa uma dependência funcional, em que, no vetor de booleano `lhs` todas as posições em `true` correspondem aos atributos presentes no lado esquerdo daquela dependência, o mesmo se aplica para o lado direito em `rhs`. Já, a `struct fd_set_t` armazena o universo de atributos `u` como um vetor de booleano e as dependências funcionais `fds` como um vetor estático de `struct fd_t` com 1000 posições, já que imaginamos que nos testes não haverá mais do que essa quantidade de dependências funcionais.

2.3 Leitura dos arquivos .fds

Na função `parse_fds_file` o arquivo é lido caractere a caractere, quando "U"é encontrado a função `reading_u` é chamada, lê os atributos e os salva no vetor de booleano `u` da `struct fd_set_t` na posição correspondente à letra em ordem alfabética. Com o universo de atributos salvo na struct, a função `reading_f` é chamada para ler as dependências funcionais e salvar no vetor `fds` da `struct fd_set_t`. Nessa função, enquanto não houver "→"os atributos lidos são colocados no vetor que corresponde ao lado esquerdo da dependência, após a flecha, lemos os atributos do lado direito, a cada ","lido, o código entende que vamos ler o lado esquerdo novamente, isso é feito até lermos todas as dependências.

2.4 Limitações da Implementação

Na implementação, os vetores que contém os atributos (`lhs`, `rhs`, `u`) tem o tamanho limitado em 26, já que consideramos apenas letras isoladas. Já o conjunto de dependências funcionais é um vetor estático `fds` de tamanho 1000, ou seja, os testes devem ter até essa quantidade de dependências.

Por ser usado valores constantes para representar os possíveis atributos de cada dependência funcional, a análise de complexidade de cada algoritmo ficou mais simplista. Se a quantidade de atributos fosse variável e crescesse assim como cresce o tamanho do conjunto F, as análises feitas mudariam.

Por fim, outra limitação de implementação é a verificação da 1FN, a implementação assume que está na 1FN, pois da forma que foi definida as estruturas de dados não é possível testar pela definição se as dependências funcionais respeitam a norma.

3. Descrição dos Algoritmos

3.1 Fecho de Atributos

O algoritmo para o cálculo do fecho recebe como parâmetros o universo de atributos U e o conjunto de dependências funcionais F, representados pela `struct fd_set_t sys`, o atributo que desejamos calcular o fecho (`attr`) e um ponteiro para o conjunto de atributos que representará o fecho (`attr_clos`). Inicialmente, verificamos se `sys` e `attr_clos` existem, se sim, para cada atributo em `attr` incluímos a letra correspondente em `attr_clos`, ou seja, adicionamos o atributo unitário no fecho. Em seguida, atribuímos ao controlador `new_attr` falso e, para cada dependência funcional em F, verificamos se a letra no lado esquerdo (lhs) está ou não no fecho (`if (sys->fds[i].lhs[j] && !attr_clos[j])`), por exemplo, se no fecho temos $\{A\}$ e a dependência é $BD \rightarrow E$, então `lhs_valid = false`, portanto pulamos essa iteração. Caso `lhs_valid = true`, para cada atributo do lado direito (rhs) que não está no fecho (`if (sys->fds[i].rhs[j] && !attr_clos[j])`) nós o adicionamos no fecho (`attr_clos[j] = true`) e mantemos o loop externo para verificar novamente todos os atributos (`new_attr = true`). O loop externo encerra quando, na última passagem por todas as dependências funcionais, não há nenhum atributo do lado direito que não esteja no fecho, ou seja, não entra em (`if (sys->fds[i].rhs[j] && !attr_clos[j])`) e, portanto, como `new_attr = false`, o loop termina.

3.2 Cobertura Mínima

O algoritmo da cobertura mínima é dividido em três passos: decompor o lado direito para ficar atributos unitários, remover os atributos redundantes do lado esquerdo e remover as dependências funcionais redundantes. O código recebe como parâmetro o universo de atributos U e o conjunto de dependências funcionais F, representados pela `struct fd_set_t sys` e outra struct desse tipo `fd_mincover` que será a cobertura mínima. A descrição de cada passo segue abaixo:

- **Passo 1:** Para cada dependência funcional, verifica-se quais letras estão do lado direito (rhs), se a letra estiver (`if(sys->fds[i].rhs[j])`) criamos uma nova dependência funcional (primeiro `for` dentro do `if`) que conterá o lado esquerdo (lhs) na íntegra (segundo for dentro do if) e o lado direito unitário (`fd_mincover->fds[idx].rhs[j] = true`);
- **Passo 2:** Para cada dependência funcional, resultante do passo 1, contamos quantas letras temos do lado esquerdo (segundo `for`) e, se for mais que uma (`if(count_lhs_attr >1)`) verificamos se há redundância. Para verificar redundância, observamos cada letra presente no lado esquerdo (`if(fd_mincover->fds[i].lhs[k])`) e calculamos o fecho ignorando esse atributo, caso o lado direito esteja no fecho (`if(fd_mincover->fds[i].rhs[l] && attr_clos[l])`) então o atributo do lado esquerdo é redundante e pode ser retirado (`fd_mincover->fds[i].lhs[k] =false`);
- **Passo 3:** Para cada dependência funcional, resultandte do passo 2, criamos uma dependência funcional temporária (`fd_t temp_fd = fd_mincover->fds[i]`), salvamos o lado esquerdo da dependência a ser retirata (`lhs[idx++] = 'A' +j`), ignoramos a dependência funcional (`fd_mincover->fds[i].lhs[j] = false; fd_mincover->fds[i].rhs[j] = false`) e calculamos o fecho, se o lado direito da dependência retirada ainda estiver no fecho significa que a dependência funcional é redundante e portanto deve ser retirada. Ao retirarmos essa dependência, devemos "puxar"as demais "para trás"(bloco do `else`), caso a dependência não seja redundante, devolvemos a dependência (`fd_mincover->fds[i] = temp_fd`).

3.3 Chaves Candidatas

O algoritmo para calcular a quantidade de chaves candidatas espera receber como parâmetros `sys`, uma estrutura que contém o conjunto universo de atributos U e o conjunto de dependências funcionais F, além do vetor de chaves em que serão inseridas as chaves candidatas encontradas para poderem ser mostradas ao final do cálculo. Ao término da função, ela retorna a quantidade de chaves candidatas encontradas, ou -1 em caso de erro.

A execução do algoritmo pode ser dividida em dois passos para melhor leitura, sendo estes os passos: separação dos atributos essenciais e busca pelas chaves candidatas. A explicação de cada passo será detalhada a seguir:

- **Passo 1:** Para cada atributo `i` pertencente ao conjunto U, é verificado se este é um atributo essencial, ou seja, um atributo que não é determinado por nenhum outro atributo. Para isso, o algoritmo checa se existe ao menos uma dependência funcional em que o atributo `i` pertença ao lado direito sem pertencer ao lado esquerdo da mesma dependência. Caso essa condição seja satisfeita em alguma dependência, o atributo `i` é determinado por outros atributos, logo não é essencial e é armazenado no vetor `remaning_idx` para futuramente testar se o atributo faz parte de alguma chave candidata. Por outro lado, se o atributo nunca aparecer no lado direito de uma dependência não-trivial, ele é classificado como essencial e obrigatoriamente precisará estar em todas as chaves candidatas encontradas a seguir.
- **Passo 2:** Após o cálculo dos essenciais, são buscadas todas as chaves candidatas existentes. Inicialmente é verificado se os próprios atributos essenciais formam uma superchave. Caso formem, então estes são a única chave candidata possível, pois pela definição, uma chave candidata é uma superchave minimal, portanto incluir qualquer outro atributo violaria a propriedade de minimalidade. Em seguida, são copiados os atributos da

chave encontrada para `keys` e é retornado o total de chaves encontradas, ou seja, o valor 1. Entretanto, caso os essenciais não formem uma superchave, então é iniciado um laço para procurar as chaves candidatas a partir dos essenciais e incluir os atributos não-essenciais. Nessa etapa, são testados subconjuntos de todos os tamanhos dos atributos restantes, utilizando a função auxiliar `generate_combinations`, a qual utiliza *backtracking* para verificar todos os subconjuntos existentes. Portanto, as chaves encontradas nessa etapa de *backtracking* terão tamanho de $|E| + 1$ a $|U|$, em que E é o conjunto de atributos essenciais. Após a geração e verificação de todos os subconjuntos, é retornada a quantidade de chaves candidatas encontradas.

Antes de iniciar o detalhamento do algoritmo de verificação de normalização, será explicada a lógica das funções auxiliares usadas, iniciando pela função `generate_combinations`. Como dito anteriormente, ela utiliza *backtracking* para gerar todas as combinações de subconjuntos existentes. Ela espera receber o mesmo parâmetro `sys`, além dos parâmetros:

- `remaning_idx`: o vetor de atributos não-essenciais a serem testados;
- `num_remaning`: a quantidade de elementos do vetor acima;
- `start_idx`: o índice do atributo inicial a ser incluído junto dos essenciais em um dado passo;
- `chosen_count`: a quantidade de elementos inseridos junto dos essenciais;
- `target_size`: o tamanho do subconjunto desejado, controlado pelo laço na função principal;
- `test_set`: o conjunto em que são incluídos os atributos para procurar as chaves candidatas;
- `keys`: a matriz em que são inseridos os atributos das chaves candidatas encontradas;
- `*num_keys`: ponteiro para a quantidade de chaves candidatas encontradas.

O caso base acontece quando a quantidade de elementos restantes inseridos no conjunto `test_set` é igual ao tamanho desejado naquele momento (`target_size`). Dentro do caso base, é verificado se alguma chave candidata está contida em `test_set` chamando a função `has_prev_key()`, ou seja, a função verifica se o conjunto testado não é um superconjunto de alguma chave anterior, além de checar se os atributos de `test_set` formam uma superchave. Caso `test_set` não contenha nenhuma chave candidata anterior e ele seja uma superchave, então foi encontrada uma nova chave candidata e em seguida seus atributos são copiados para `keys` e é incrementada a quantidade de chaves candidatas. Por fim, encerra-se a recursão.

Por sua vez, o passo recursivo testará todas as combinações possíveis a partir dos atributos que não foram classificados como essenciais. Esse teste acontece dentro de um laço, iniciando em `start_idx` e indo até `num_remaning - 1`. Antes de iniciar a recursão, é incluído temporariamente o atributo `remaning_idx[i]` em `test_set`, e em seguida a própria função é chamada, passando como novos parâmetros `i + 1` para `start_idx` e `chosen_count + 1` para `chosen_count`, evitando repetição de elementos. Após sair da recursão, o atributo anteriormente incluído é retirado para o próximo teste.

Por fim, as outras duas funções usadas possuem algoritmos mais diretos. A função `is_superkey()` calcula o fecho de `test_set` para em seguida checar se este representa todo o conjunto universo U, ou seja, verifica se $test_set^+ = U$. Já a função `has_prev_key` itera sobre as chaves já armazenadas em `keys` (até o limite indicado por `num_keys`) e verifica se alguma delas é subconjunto de teste ($K \subset test_set$), assim garantindo a propriedade de minimalidade das novas chaves candidatas.

3.4 Verificação BCNF e 3FN

O algoritmo para verificar as normas 3FN e BCNF espera receber apenas o parâmetro `sys`, o qual contém o conjunto universo U e o conjunto de dependências funcionais F. Ao término da função, será retornado `OK` em caso de sucesso ou `ERROR` em caso de parâmetro nulo.

O algoritmo pode ser dividido em 4 passos para melhor leitura, sendo estes: cálculo de chaves candidatas, verificação de atributos primos, verificação de violação das normas e impressão das possíveis violações junto de seus motivos. A explicação de cada passo está a seguir:

- **Passo 1:** São salvas todas as chaves candidatas na matriz `keys` junto de sua quantidade armazenada em `num_keys`, a partir de `sys`, chamando a função `candidate_keys`, detalhada na seção anterior.
- **Passo 2:** É feita a busca pelos atributos primos, ou seja, atributos que estão em alguma chave candidata (`keys[i][j]`). Cada atributo primo é armazenado no vetor `is_prime`.
- **Passo 3:** Antes da iteração, são definidas as *flags* `has_bcnf_violated` e `has_tnf_violated`, ambas `false` inicialmente, e os vetores `bcnf_violated` e `tnf_violated`, nos quais serão armazenadas quais dependências funcionais violaram as normas. Em seguida é iniciado um laço que percorrerá todas as dependências funcionais em F, a fim de procurar violações da 3FN e BCNF. Primeiramente é feita a busca pela existência de dependências funcionais triviais, ou seja, aquelas em que $Y \subseteq X$ para alguma $X \rightarrow Y$. Para isso, o algoritmo verifica se existe algum atributo pertencente exclusivamente ao lado direito da dependência. Caso a verificação falhe, então a DF `i` é trivial, logo não é possível violar nenhuma norma. Por outro lado, caso a DF seja não-trivial, então ela pode violar alguma das duas normas, começando pela BCNF, que será violada caso o lado esquerdo dessa DF não seja uma superchave. Além de violar a BCNF, a 3FN poderá ser violada também, para isso é iniciado outro laço que percorre todos os atributos existentes e caso o atributo `j` da DF `i` esteja apenas no lado direito e não seja primo, então a 3FN foi violada. Em ambas as violações, a respectiva *flag* de controle é

ativada e a DF `i` é guardada para justificar a violação posteriormente.

- **Passo 4:** Este passo é dedicado à impressão de resultados, informando se houve violações das normas ou não e, caso tenham ocorrido violações, são mostradas as dependências funcionais que as violaram. Além disso, é feito a decomposição do lado direito para cada impressão.

4. Análise de Complexidade

4.1 Fecho de Atributos

Considerando N o número de dependências funcionais, A = MAX_ATTRS = 26 e K o tamanho de `attr`, vamos analisar o a complexidade do algoritmo

```
for (int i = 0; attr[i] != '\0'; i++) { //
    roda no maximo K vezes
    attr_clos[attr[i] - 'A'] = true;
}
```

Então, a complexidade desse trecho é $O(K)$.

```
while (new_attr ) { //roda enquanto encontrar
    um novo atributo, no maximo roda A vezes
    for (int i = 0; i < sys->size; i++) { //
        roda N vezes
        bool lhs_valid = true;
        for (int j = 0; j < MAX_ATTRS; j++) {
            //roda A vezes, no maximo
            if (sys->fds[i].lhs[j] && !
                attr_clos[j]) {
                lhs_valid = false;
                break;
            }
        }
        if (lhs_valid) {
            for (int j = 0; j < MAX_ATTRS; j
                ++) { //roda A vezes, no
                maximo
                if (sys->fds[i].rhs[j] && !
                    attr_clos[j]) {
                    attr_clos[j] = true;
                    new_attr = true;
                }
            }
        }

    }
}
```

No loop interno ao `while`, se considerarmos o pior caso, ou seja, que sempre entraremos no `if(lhs_valid)`, teremos o custo $O(N * (A + A)) = O(N * 2A)$, já, no loop externo (`while`) temos, no máximo o custo $O(A)$, se colocarmos todas as letras dentro de `attr_clos`. Unindo todos os custos, temos $O(N * 2A) * O(A) + O(K)$, como A é constante e sabendo que K é limitado por 26, então o custo total é $O(N)$.

4.2 Cobertura Mínima

Considerando N o número de dependências funcionais, A = MAX_ATTRS = 26, N' a quantidade de dependências após um passo e C a complexidade do fecho, vamos analisar cada passo.

Passo 1:

```
for(int i = 0; i < sys->size; i++){ // roda N
    vezes
    for(int j = 0; j < MAX_ATTRS; j++){ //
        roda A vezes
        if(sys->fds[i].rhs[j]){
            for(int k = 0; k < MAX_ATTRS; k++)
                // roda A vezes
            for(int k = 0; k < MAX_ATTRS; k++)
                // roda A vezes
            // ou seja, A+A = 2A
```

Considerando o pior caso, ou seja, que sempre entraramos no `if(sys->fds[i].rhs[j]`, a complexidade é $O(N * A * 2A) = O(N * 2A) = O(N)$, já que A é uma constante. Além disso, o tamanho do conjunto de dependências funcionais pode no máximo ser $N' = N * A$, ou seja, se para cada dependência funcional temos do lado direito todas as letras do alfabeto, então geraremos $N * A$ novas dependências funcionais com os lados direitos unitários.

Passo 2:

```
for(int i = 0; i < fd_mincover->size; i++){ //
     roda N' vezes
    for(int k = 0; k < MAX_ATTRS; k++) // roda
         A vezes
    if(count_lhs_attr > 1){
        for(int k = 0; k < MAX_ATTRS; k++){ //
             roda A vezes
            if(fd_mincover->fds[i].lhs[k]){
                for(int m = 0; m < MAX_ATTRS;
                    m++) // roda A vezes
                closure(fd_mincover, lhs_wo_k,
                     attr_clos); // O(C)
                for(int l = 0; l < MAX_ATTRS;
                    l++) // roda A vezes
                // ou seja 2A + O(C)
            }
        } // ou seja A*(2A + O(C))
    }
}
```

Considerando que em todas as dependências funcionais teremos todas as letras do alfabeto como atributos, ou seja, sempre entraremos em `count_lhs_attr` e em `if(fd_mincover->fds[i].lhs[k])`, a complexidade é $O(N' * A) + O(N' * A * (2A + C)) = O(N * A * A) + O((N*A)*A*(2A+C)) = O(N*A^2)+O(N*2A^3+N*A^2*C) = O(N) + O(N + N * N) = O(N) + O(N) = O(N^2)$. O tamanho do conjunto de dependências funcionais resultante pode no máximo ser $N'' = N * A$, pois apenas retiramos, ou não, atributos redundantes do lado esquerdo.

Passo 3:

```
for(int i = 0; i < fd_mincover->size; i++){ //
     roda N'' vezes
    for(int j = 0; j < MAX_ATTRS; j++) // roda
         A vezes
    for (int j = 0; j < MAX_ATTRS; j++) //
        roda A vezes
    closure(fd_mincover, lhs, attr_clos); // O
        (C)
    for(int l = 0; l < MAX_ATTRS; l++) // roda
         A vezes
```

```
    // se for redundante, desloca todo o resto
        das dependencias
    for (int k = i; k < fd_mincover->size - 1;
        k++) { // O(N'') no pior caso
        fd_mincover->fds[k] = fd_mincover->fds
            [k + 1];
    }
}
```

Considerando que em todas as iterações teremos dependências irrelevantes, exceto na última, pois é necessário ter pelo menos uma dependência funcional, a complexidade é $O(N'' * (3A + O(C) + (O(N'') - 1)))$ = $O(N * A * (3A + O(N) + (O(N * A) - 1)))$ = $O(N * (O(N) + O(N) - 1))$ = $O(N * N)$ = $O(N^2)$.

Portanto, unindo os três passos, a complexidade final é $O(N) + O(N^2) + O(N^2)$ = $O(N^2)$.

4.3 Chaves Candidatas

Primeiramente, definem-se as seguintes variáveis: N como a quantidade de dependências funcionais ($|F|$), A como a quantidade de atributos (constante com MAX_ATTRS $= 26$), E como a quantidade de atributos classificados como essenciais ($0 \leq E \leq A$), K_{ant} como a quantidade de chaves candidatas encontradas e R como a quantidade de atributos não-essenciais dada por $R = A - E$ ($0 \leq R \leq A$).

Inicialmente, antes de analisar a complexidade da função principal, será analisada a complexidade das funções auxiliares `is_superkey` e `has_prev_key`.

is_superkey:

```
// Converte test_set para string de atributos
for (int i = 0; i < MAX_ATTRS; i++){
    // roda A vezes
}

// Complexidade do calculo do fecho
closure(sys, attr_str, attr_closure);

// Verifica se eh superchave
for (int i = 0; i < MAX_ATTRS; i++){
    // roda A vezes
}
```

O primeiro laço executa A vezes, logo a sua complexidade é dada por $O(A)$. O cálculo do fecho, como já mostrado na sua própria seção, tem complexidade $O(N)$. Por fim, o último laço roda A vezes também, portanto $O(A)$. Assim, juntando as três informações, concluímos que a complexidade do algoritmo é $O(A) + O(N) + O(A)$, como A é uma constante, podemos simplificar para $O(N)$. Vale destacar que a complexidade seria dada por $O(N \cdot A^2)$ se A não fosse constante. Esse valor será usado a seguir para explicar a complexidade final teórica se a quantidade de atributos fosse variável.

has_prev_key:

```
for (int i = 0; i < num_keys; i++){ // itera
    sobre as chaves candidatas
    for (int j = 0; j < MAX_ATTRS; j++){
        // roda A vezes
    }
}
```

A função possui um único bloco de laço. O laço mais externo executará K_{ant} vezes e o mais interno A vezes, assim a complexidade da função é dada por $O(K_{\text{ant}} \cdot A)$, simplificando, temos $O(K_{\text{ant}})$.

Analisando a função principal, assim como na explicação do algoritmo ela foi dividida em dois passos, na análise de custo será vista a complexidade de cada passo, agrupando os casos intermediários ao final da explicação para a complexidade do algoritmo como um todo.

Passo 1:

```
// Separa atributos essenciais dos restantes
for (int i = 0; i < MAX_ATTRS; i++){ // roda A
     vezes
    if (sys->u[i]){

        for (int j = 0; j < sys->size; j++){
            // roda N vezes
        }

        // [...]
    }
}
```

Nesse primeiro momento, para calcular os atributos essenciais, o laço mais externo rodará A vezes, uma vez para cada atributo. O mais interno, se existir ao menos um atributo no conjunto universo, rodará N vezes, assim a complexidade desse primeiro passo é dada por $O(A \cdot N)$, ou, simplificando, $O(N)$.

Passo 2:

```
// Testa se essentials forma uma superchave
if (has_essentials && is_superkey(sys,
    essentials)){
    // Melhor caso
}

// Testa combinacoes dos atributos restantes
for (int tam = 1; tam <= num_remaning; tam++){
    // Caso geral e pior caso
    generate_combinations(sys, remaning_idx,
        num_remaning, 0, 0, tam, test_set,
        keys, &num_keys);
}
```

Neste passo, é possível separar a análise em dois casos: o melhor caso, verificado se a primeira condicional for satisfeita, e o caso geral, que surge dentro do laço, podendo chegar ao pior caso. Para a análise do melhor caso, basta verificar a complexidade da `is_superkey`, a qual já foi calculada anteriormente, assim a complexidade do bloco é dada por $O(N)$. Caso o primeiro bloco não ocorra, então o bloco que contém o *backtracking* acontecerá. Nesse cenário, é necessário analisar quantos nós a recursão gera e a complexidade de cada nó. Dado um tamanho fixo `target_size` = k, a função `generate_combinations` percorre uma árvore de combinações simples de R elementos tomados k a k, gerando $\binom{R}{k}$ folhas, e, somando todas as iterações, temos $\sum_{k=1}^{R} \binom{R}{k}$. Expandindo a soma pela identidade binomial, obtém-se $2^R - 1 \leq 2^A$. Para cada folha, são chamadas as funções `has_prev_key` e `is_superkey`, as quais pos-

suem complexidade $O(K_{\text{ant}} \cdot A)$ e $O(N \cdot A^2)$, respectivamente. Analisando o pior caso possível, ou seja, o caso em que todas as combinações são testadas e as chaves candidatas estão concentradas em subconjuntos maiores, o algoritmo visitará uma quantidade de nós proporcional a 2^R. Portanto, a complexidade do pior caso é dada por $\sum_{\text{folhas}}(O(K_{\text{ant}} \cdot A) + O(N \cdot A^2)) = O(2^R \cdot N \cdot A^2)$. Como no algoritmo A é uma constante, a complexidade pode ser simplificada para $O(N)$.

Portanto, a partir do Passo 1 e do Passo 2, conclui-se que as complexidades são dadas por $O(N)$. Assim, a complexidade geral de `candidate_keys` é $O(N)$. É importante destacar que isso é possível graças ao fato de A ser uma constante, caso não fosse, a complexidade do algoritmo seria dada por $O(2^R \cdot N \cdot A^2)$, pois o problema de encontrar chaves candidatas é $\mathcal{NP}$-difícil.

4.4 Verificação BCNF e 3FN

Primeiramente, definem-se as seguintes variáveis: N como a quantidade de dependências funcionais ($|F|$), A como a quantidade de atributos (constante com MAX_ATTRS $= 26$) e K como a quantidade de chaves candidatas existentes.

Assim como na seção anterior, será feita a análise de custo de cada passo, juntando os resultados intermediários ao fim.

Passo 1:

```
int num_keys = candidate_keys(sys, keys);
```

Nesse trecho, calcula-se a quantidade de chaves candidatas. Como já discutido na seção anterior, o custo de `candidate_keys` para esse algoritmo é dado por $O(N)$.

Passo 2:

```
// Verifica atributos primos
for (int i = 0; i < num_keys; i++){ // roda K
    vezes
    for (int j = 0; j < MAX_ATTRS; j++){
        // roda A vezes
    }
}
```

Para a busca dos atributos primos, são executados dois laços aninhados. Os laços externo e interno rodarão, respectivamente, K e A vezes; logo, o custo desse trecho é dado por $O(K \cdot A)$, ou, simplificando, $O(K)$.

Passo 3:

```
// Verifica violacoes de 3FN e BCNF
for (int i = 0; i < sys->size; i++){ // roda N
     vezes
    // Verifica existencia de DFs triviais
    for (int j = 0; j < MAX_ATTRS; j++){
        // roda A vezes
    }

    // Violacoes das normas
    // Violacao da BCNF pela definicao
    if (!is_trivial && !(is_superkey(sys, sys
        ->fds[i].lhs))){
        for (int j = 0; j < MAX_ATTRS; j++){
            // roda A vezes
        }
    }
}
```

Para esse trecho, o laço mais externo rodará N vezes. O primeiro laço interno será executado A vezes. Para o segundo bloco, o qual contém a checagem de existência de DF trivial e a chamada da função `is_superkey`, a qual possui complexidade $O(N)$ como já vista anteriormente, será considerado o pior cenário possível, ou seja, todas as DFs são não-triviais (além dos lados esquerdos não serem superchaves); assim, o algoritmo entrará no bloco todas as iterações. Nesse bloco há mais um laço que também será executado A vezes. Portanto, a complexidade desse passo é dada por $O(N \cdot (A + N + A)) = O(N^2 + N \cdot A)$, tomando o termo dominante, $O(N^2)$.

Passo 4:

```
for (int i = 0; i < sys->size; i++){ roda N
    vezes
    if (bcnf_violated[i]){
        // Decomposicao do lado direito
        for (int j = 0; j < MAX_ATTRS; j++){
            // roda A vezes
            if (sys->fds[i].rhs[j] && !sys->
                fds[i].lhs[j]){
                    printf("VIOLATION␣BCNF:␣")
                        ;
                    print_attrs(sys->fds[i].
                        lhs);
                    printf("->%c␣(", 'A' + j);
                    print_attrs(sys->fds[i].
                        lhs);
                    printf("␣not␣superkey)\n")
                        ;
            }
        }
    }
}

for (int i = 0; i < sys->size; i++){ roda N
    vezes
    if (tnf_violated[i]){
        // Decomposicao do lado direito
        for (int j = 0; j < MAX_ATTRS; j++){
            // roda A vezes
            if (sys->fds[i].rhs[j] && !sys->
                fds[i].lhs[j] && !is_prime[j])
                {
                    printf("VIOLATION␣3FN:␣");
                    print_attrs(sys->fds[i].
                        lhs);
                    printf("->%c␣(%c␣not␣prime
                        ,␣", 'A' + j, 'A' + j)
                        ;
                    print_attrs(sys->fds[i].
                        lhs);
                    printf("␣not␣superkey)\n")
                        ;
            }
        }
    }
}
```

Para esse trecho, como os dados laços são essencialmente iguais, será considerado o pior cenário possível, ou seja, existe violação da 3FN e BCNF em todas as iterações, assim a complexidade de um laço será refletido no outro. O laço

mais externo rodará N vezes e o mais interno A vezes. A complexidade da função auxiliar `print_attrs` é $O(A)$, pois executa um laço que percorre a quantidade de atributos A para imprimi-los. Portanto, a complexidade de cada laço é dada por $O(N \cdot A \cdot A) = O(N \cdot A^2)$. Como A é constante, é possível simplificar para $O(N)$. Portanto, a complexidade do Passo 4 é dada por $O(N) + O(N) = O(N)$

Assim, a complexidade de `normal_form` será, juntando todos os resultados dos passos, $O(N) + O(K) + O(N^2) + O(N)$. Como K é a quantidade de chaves candidatas e foi definido um conjunto finito de atributos, as possibilidades de chaves candidatas são limitadas por uma constante finita; logo, o termo de maior ordem será $O(N^2)$, sendo esta a complexidade do algoritmo como um todo.

5. Estudos de Caso

5.1 Fecho de Atributos

O exemplo `closure_03.fds`, o qual possui como entrada $U = \{A, B, C, D, E\}$ e $F = \{A \to B, C \to D, BD \to E\}$ e que pede o cálculo do fecho de A necessita passar pelo loop externo mais de uma vez e A não é superchave:
Inicialização: Iniciamos o fecho com o próprio atributo de origem: $A^+ = \{A\}$

Primeira Iteração do Loop Externo: O algoritmo percorre todas as dependências funcionais em F:

1) Para $A \to B$: O lado esquerdo é $\{A\}$. Como $A \in A^+$, o LHS é válido. O atributo B não está no fecho, portanto ele é adicionado $A^+ = \{A, B\}$
2) Para $C \to D$: O lado esquerdo é $\{C\}$. Como $C \notin A^+$, o LHS é inválido e a dependência é ignorada nessa iteração.
3) Para $BD \to E$: O lado esquerdo é $\{B, D\}$. Embora $B \in A^+$, o atributo $D \notin A^+$, logo o LHS é inválido e a dependência é ignorada.

Segunda Iteração do Loop Externo: Percorremos novamente todo o conjunto F:

1) Para $A \to B$: $A \in A^+$, mas B já pertence ao fecho. Nenhuma alteração é feita.
2) Para $C \to D$: $C \notin A^+$, LHS inválido. Nenhuma alteração.
3) Para $BD \to E$: $D \notin A^+$, LHS inválido. Nenhuma alteração.

Como nenhum novo atributo foi descoberto nesta passagem, a variável `new_attr` permanece `false` e o loop externo encerra, logo o fecho resultante é: $A^+ = \{A, B\}$.

5.2 Cobertura Mínima

O exemplo `mincover_09.fds`, o qual possui como entrada $U = \{A, B, C, D, E\}$ e $F = \{A \to BC, B \to C, AB \to D, ACD \to E\}$, passa pelos três passos.

Passo 1: Separamos todos os atributos do lado direito, no caso apenas a dependência $\{A \to BC\}$, passando a ter $F' = \{$ A $\to B, A \to C, B \to C, AB \to D, ACD \to E\}$;

Passo 2: Verificamos se há redundância nos atributos do lado esquerdo para as dependências $AB \to D$ e $ACD \to E$:

- Para $AB \to D$:
  - Testamos se A é redundante calculando $B^+ = \{B, C\}$. Como $D \notin B^+$, o atributo A é necessário.
  - Testamos se B é redundante calculando $A^+ = \{A, B, C, D, E\}$. Como $D \in A^+$, o atributo B é redundante e pode ser removido do LHS.
  - A dependência $AB \to D$ é simplificada para $A \to D$.
- Para $ACD \to E$:
  - Testamos se A é redundante calculando $(CD)^+ = \{C, D\}$. Como $E \notin (CD)^+$, A é necessário.
  - Testamos se C é redundante calculando $(AD)^+ = \{A, B, C, D, E\}$. Como $E \in (AD)^+$, o atributo C é redundante e pode ser removido do LHS.
  - A dependência passa a ser $AD \to E$.
  - Em seguida, testamos se D é redundante em $AD \to E$ calculando $A^+ = \{A, B, C, D, E\}$. Como $E \in A^+$, o atributo D também é redundante no LHS.
  - A dependência é simplificada para $A \to E$.

Portanto, $F'' = \{A \to B, A \to C, B \to C, A \to D, A \to E\}$.

Passo 3: Verificamos se há redundância em alguma dependência funcional:

- Para $A \to B$: calculamos A^+ sem a dependência $\{A \to B\}$, obtendo $A^+ = \{A, C, D, E\}$. Como $B \notin A^+$, $A \to B$ não é redundante.
- Para $A \to C$: calculamos A^+sem a dependência $\{A \to C\}$, obtendo $A^+ = \{A, B, C, D, E\}$. Como $C \in A^+$, a dependência $A \to C$ é redundante e deve ser removida.
- Para $B \to C$: calculamos B^+ sem a dependência $\{B \to C\}$, obtendo $B^+ = \{B\}$. Como $C \notin B^+$, $B \to C$ não é redundante.
- Para $A \to D$: calculamos A^+ sem a dependência$\{A \to D\}$, obtendo $A^+ = \{A, B, C, E\}$. Como $D \notin A^+$, $A \to D$ não é redundante.
- Para $A \to E$: calculamos A^+ sem a dependência $\{A \to E\}$, obtendo $A^+ = \{A, B, C, D\}$. Como $E \notin A^+$, $A \to E$ não é redundante.

Dessa forma, a cobertura mínima resultante F_c é:$F_c = \{A \to B, B \to C, A \to D, A \to E\}$

5.3 Chaves Candidatas

O exemplo `key_05.fds`, o qual possui como entrada $U = \{A, B, C, D\}$ e $F = \{AB \to C, C \to D, D \to B\}$, passa por todas as etapas do algoritmo.

Passo 1: A busca pelos atributos essenciais é iniciada. O algoritmo identifica que os atributos A, B, C e D estão em U e que apenas o atributo A não aparece no lado direito de nenhuma dependência funcional, classificando-o como essencial e os demais atributos como não-essenciais, armazenados no vetor `remaning_idx` e com `num_remaning` assumindo o valor 3.

Passo 2.1: Como A foi classificado como essencial, a *flag* `has_essentials` termina o laço do Passo 1 como `true`. Dessa forma, a função `is_superkey` é chamada para verificar se A é uma superchave. Dentro da função, é

calculado A^+. Como o fecho de A é apenas o próprio atributo A ($A^+ = \{A\}$), no momento de verificar se A^+ é igual ao conjunto U, a função encerra retornando `false`.

Passo 2.2: Uma vez que A não é superchave, inicia-se a fase de testar todas as combinações com os elementos não-essenciais a fim de procurar as chaves candidatas utilizando o algoritmo de *backtracking*, com o laço iniciando com subconjuntos de tamanho 1 e terminando com subconjuntos de tamanho 3. Para melhor entendimento, considere a tabela abaixo:

Tabela I
RASTREAMENTO DA BUSCA COMBINATÓRIA PARA `KEY_05.FDS`.

tam	Combinação	`!has_prev_key`	Fecho	Chave?
1	$\{A, B\}$	`true`	$\{A, B, C, D\}$	Sim
1	$\{A, C\}$	`true`	$\{A, B, C, D\}$	Sim
1	$\{A, D\}$	`true`	$\{A, B, C, D\}$	Sim
2	$\{A, B, C\}$	`false`	—	Não
2	$\{A, B, D\}$	`false`	—	Não
2	$\{A, C, D\}$	`false`	—	Não
3	$\{A, B, C, D\}$	`false`	—	Não

É possível observar na tabela que foram incluídas três novas chaves, ou seja, a variável `num_keys` termina a execução com o valor 3. Além disso, quando se iniciaram os testes para subconjuntos de tamanhos 2 e 3, o fecho não foi calculado, pois a função `has_prev_key` identificou que as combinações de atributos já continham outras chaves candidatas vistas em passos anteriores, economizando processamento e preservando a propriedade de minimalidade das chaves candidatas. Ao término da função, é retornado o valor 3, a quantidade de chaves candidatas encontradas, enquanto as chaves são armazenadas na matriz `keys`, sendo elas: $\{A, B\}$, $\{A, C\}$ e $\{A, D\}$.

5.4 Verificação BCNF e 3FN

Para o estudo de caso desta seção, serão apresentados dois exemplos: um para demonstrar a violação de ambas as formas normais e outro para demonstrar exclusivamente a violação da BCNF.

- **Violação de Ambas as Formas:** O exemplo `normalform_03.fds`, o qual possui como entrada $U = \{A, B, C\}$ e $F = \{A \to B, B \to C\}$, executa as seguintes etapas do algoritmo:
  Passo 1: Realiza-se a busca pelas chaves candidatas. O algoritmo retorna que existe apenas uma chave candidata, sendo esta a chave $\{A\}$.
  Passo 2: Identificam-se os atributos primos. A partir do Passo 1, é possível observar que o único atributo primo encontrado é o próprio A.
  Passo 3: Inicia-se o laço para verificar se alguma dependência funcional presente em F viola as formas normais. O primeiro passo é checar a existência de DFs triviais. Analisando o conjunto F, constata-se que não existem dependências triviais; logo, em todas as iterações a variável `is_trivial` permanece como `false`, permitindo a entrada no bloco de verificação. Para a primeira DF ($A \to B$), o lado esquerdo é uma superchave; logo, nenhuma norma é violada e o algoritmo avança para a próxima iteração. Para a segunda DF ($B \to C$), o lado esquerdo não é uma superchave, violando a BCNF. Em seguida, avalia-se a 3FN: como o lado direito (C) não é um atributo primo, a 3FN também é violada.
  Passo 4: Como ambas as formas foram violadas no Passo 3, o algoritmo exibe em tela as respectivas mensagens, justificando a violação da BCNF pelo fato de o determinante B não ser uma superchave, e a da 3FN por B não ser superchave juntamente com o atributo determinado C não ser primo.
- **Violação da BCNF:** Para este estudo de caso, será utilizado o exemplo `normalform_02.fds`, o qual possui como entrada $U = \{A, B, C\}$ e $F = \{AB \to C, C \to B\}$. Devido ao fato de sua execução ser semelhante ao exemplo anterior, a explicação dos passos será mais resumida. As seguintes etapas do algoritmo são executadas:
  Passo 1: Identificam-se as chaves candidatas $\{A, B\}$ e $\{A, C\}$.
  Passo 2: Identifica-se que todos os atributos do conjunto U são primos, ou seja, os atributos A, B e C.
  Passo 3: Observando F, é possível detectar que não existem DFs triviais. O lado esquerdo da primeira DF ($AB \to C$) é uma superchave; logo, nenhuma norma é violada. Por outro lado, o lado esquerdo da segunda DF ($C \to B$) não é uma superchave, violando a BCNF. Em seguida, ao avaliar a 3FN, o algoritmo identifica que o atributo do lado direito (B) é primo, satisfazendo a condição e não violando a norma.
  Passo 4: Como apenas a BCNF foi violada no Passo 3, o algoritmo exibe em tela a respectiva mensagem de violação, justificando com o atributo C não ser uma superchave. Em seguida, informa que a 3FN não foi violada, sinalizando com ”OK”.

6. Considerações Finais

Esse trabalho teve como embasamento os conceitos e algoritmos passados em aula, além disso, consultas pontuais à IA foram feitos para identificar alguns erros em trechos de código e gerar alguns dos exemplos. O código fonte pode ser acessado através de https://github.com/sergingameprayss/BD.